

DBI Dokumentation – Statify

Team

Dominik Nageler, Jonas Marte

1. Projektbeschreibung

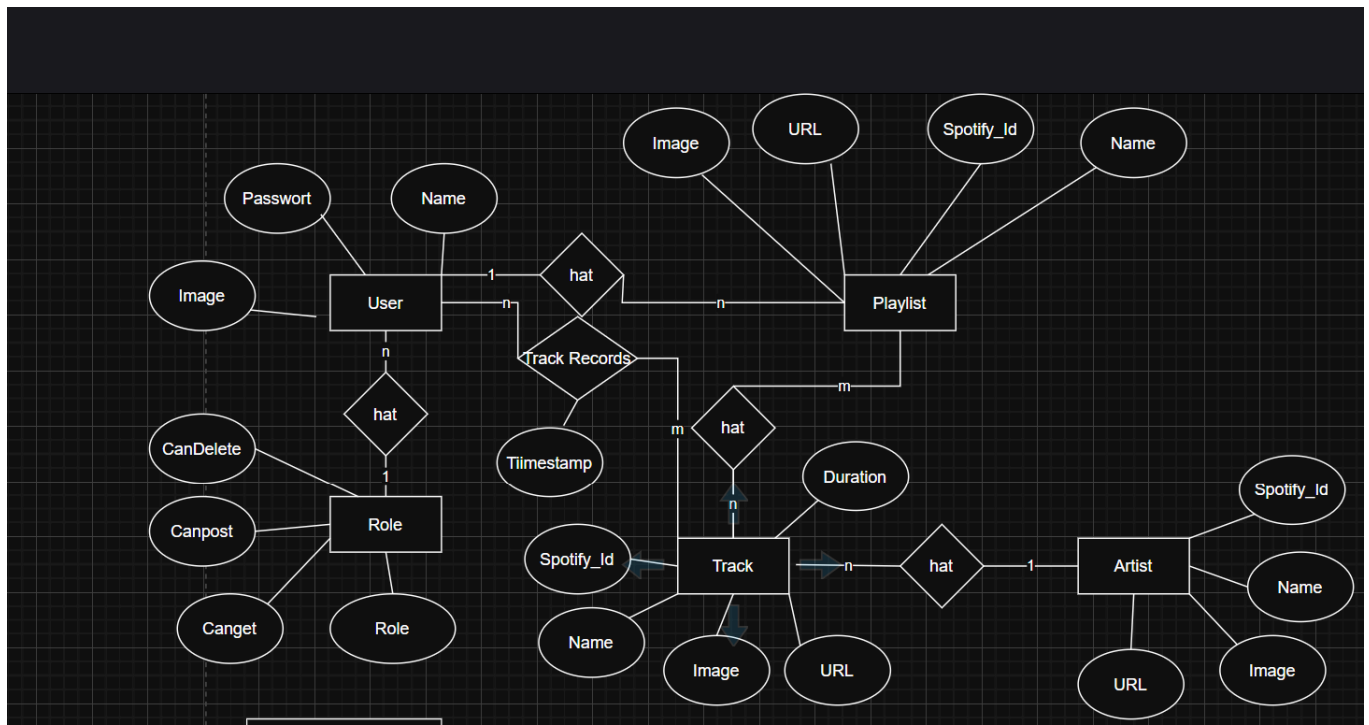
In unserer App soll man seine eigenen Spotify-Statistiken anschauen können. Dafür werden, falls vorhanden, die Daten aus unserer eigenen DB geladen. Die Daten werden immer erneuert, wenn man die App startet – sofern es neue Daten gibt. So kann man dann nach längerer Benutzung Statistiken anschauen wie:

- Anzahl angehörter Minuten
- Meistgehörte/r
 - Artist
 - Track
 - Playlist

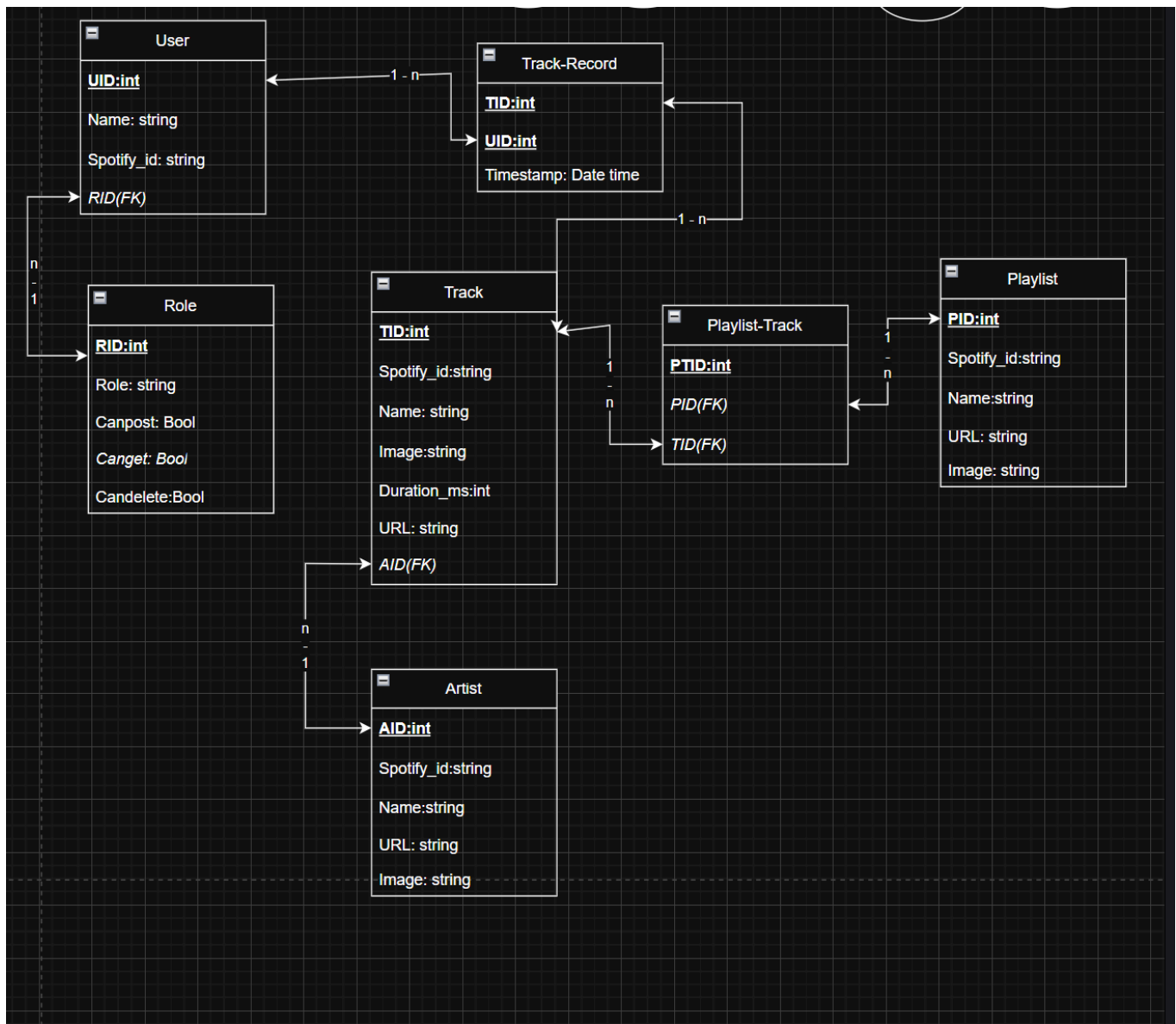
In unserem Mainwindow sieht man eine grobe Übersicht über ein paar spannende Daten. Über unsere Nav-Bar kann man genauere Daten anschauen wie meistgehörte Artists, Tracks, etc.

2. Planung

ERM



Relationales Modell



Normalformen

Alle Tabellen erfüllen die 1., 2. und 3. Normalform:

1NF

Jede Spalte enthält nur atomare (unteilbare) Werte, es gibt keine Wiederholungsgruppen und jede Zeile ist durch einen Primärschlüssel eindeutig identifizierbar. Alle unsere Tabellen sind so aufgebaut – z. B. speichert `Track_Record` pro Hörvorgang genau einen Timestamp und eine Duration, nicht eine Liste davon.

2NF

Alle Nicht-Schlüssel-Attribute sind voll funktional abhängig vom gesamten Primärschlüssel (relevant bei zusammengesetzten PKs).

3NF

Es gibt keine transitiven Abhängigkeiten zwischen Nicht-Schlüssel-Attributen. Zum Beispiel: `Follower_count` in `Artist` hängt direkt von `AID` ab, nicht von einem anderen Nicht-Schlüssel-Attribut. Berechnete Werte wie „Gesamtminuten“ werden gar nicht gespeichert, sondern per SQL aus `Track_Record` abgeleitet.

3. Umsetzung

Technologien

Technologie	Version
Python	3.11
annotated-doc	0.0.4
annotated-types	0.7.0
anyio	4.13.0
click	8.3.3
colorama	0.4.6
fastapi	0.136.1
fastapi-restful	0.6.0
greenlet	3.5.0
h11	0.16.0
idna	3.13
mypy_extensions	1.1.0
psutil	5.9.8
pydantic	2.13.3
pydantic_core	2.46.3
SQLAlchemy	2.0.49
starlette	1.0.0
typing-inspect	0.9.0
typing-inspection	0.4.2
typing_extensions	4.15.0
uvicorn	0.46.0
bcrypt	4.0.1
spotipy	2.26.0

Technologie	Version
python-dotenv	1.2.2
passlib	1.7.4

Datenbank

User

Tabelle mit allen Usern

Role

Tabelle für die Rollen, die an die User vergeben werden

Track

Tabelle, um alle Tracks von der Spotify API abzuspeichern

Track Record

Verbindungstabelle zwischen User und Track + speichert zusätzlich einen Timestamp ab

Artist Track

Verbindungstabelle zwischen Artist und Track

Artist

Tabelle, um jeden Artist von den abgespeicherten Tracks zu speichern

Playlist Track

Verbindungstabelle zwischen Playlist und Track

Playlist

Tabelle, um die Playlist abzuspeichern

API-Endpunkte

Playlist (/playlist)

Methode	Pfad	Bedeutung	Auth
POST	/playlist/sync	Playlists von Spotify synchronisieren	user, admin
POST	/playlist/sync/{playlist_id}/tracks	Tracks einer Playlist von Spotify nachladen	user, admin
GET	/playlist/{playlist_id}/tracks	Tracks einer Playlist	user, admin
GET	/playlist	Alle Playlists eines Users	user, admin
DELETE	/playlist/{playlist_id}	Playlist löschen	user, admin

Beispiel:

GET /playlist/2/tracks

Response

```
{
  "Spotify_id": "1hz7SRTGUNAtIQ46qiNv2p",
  "Name": "GONE, GONE / THANK YOU",
  "Image":
    "https://i.scdn.co/image/ab67616d0000b27330a635de2bb0caa4e26f6abb",
  "Duration": 375386,
  "URL": "https://open.spotify.com/track/1hz7SRTGUNAtIQ46qiNv2p",
  "AID": 21,
  "Id": 51
}
```

Track-Record (/track_record)

Methode	Pfad	Bedeutung	Auth
GET	/track_record	Hörverlauf mit Track-, Artist- und Playcount-Infos (JOIN)	user, admin
POST	/track_record/sync/{user_id}	Zuletzt gehörte Tracks von Spotify synchronisieren	user, admin
GET	/track_record/playtime	Holt die gesamte Playtime über Tage gefiltert	user, admin

Beispiel:

GET /track_record/

Response

```
{
  "Timestamp": "2026-06-09T18:05:47",
  "UID": 1,
  "TID": 223,
  "Id": 150,
  "Track_Name": "Reality",
  "Track_Image":
  "https://i.scdn.co/image/ab67616d0000b273014611dc4bf734a3422e6d8a",
  "Artist_Name": "Car Seat Headrest",
  "URL": "https://open.spotify.com/track/2ui8Zgp3oIfNSuEHjQYkyD",
  "Duration": 673670,
  "Playcount": 3
}
```

Artist (/artist)

Methode	Pfad	Bedeutung	Auth
GET	/artist	Artists eines Users mit Gesamt-Playtime (JOIN)	user, admin
DELETE	/artist/{artist_id}	Artist löschen	admin
GET	/artist/{artist_id}/tracks	Holt alle Tracks von einem Artist	user, admin

Beispiel:

GET /artist/

Response

```
{
  "Name": "Radiohead",
  "Spotify_id": "4Z8W4fKeB5YxbusRsdQVPb",
  "Image":
  "https://i.scdn.co/image/ab6761610000e5eb4104fbd80f1f795728abbd59",
  "URL": "https://open.spotify.com/artist/4Z8W4fKeB5YxbusRsdQVPb",
  "Id": 1,
}
```

```
"Playtime": 8970681
}
```

User (/user)

Methode	Pfad	Bedeutung	Auth
POST	/user/register	Neuen User registrieren	öffentlich
POST	/user/login	User einloggen	öffentlich
PUT	/user/	User aktualisieren	user, admin
DELETE	/user/logout	User löschen / ausloggen	user, admin

Beispiel:

POST /user/register

Response

```
{
  "Name": "beispiel",
  "Id": 7,
  "Image": "https://i.scdn.co/image/ab67757000000ee857fb7e067bd41efb125a383c4"
}
```

Track (/track)

Methode	Pfad	Bedeutung	Auth
GET	/track/tracks	Holt alle Tracks von einem User	user, admin
GET	/track/{track_id}	Holt genau einen Track	user, admin
DELETE	/track/{track_id}	Löscht einen Track	admin

Beispiel:

GET /track/5

Response

```
{
  "Spotify_id": "1bNFCane3QoFy6Yf5wddf9",
  "Name": "Unforgiving Girl (She's Not An)",
}
```



```
"Image": "https://i.scdn.co/image/ab67616d0000b2734094272fe172643edfdbba48",
"Duration": 326427,
"URL": "https://open.spotify.com/track/1bNFcAne3QoFy6Yf5wddf9",
"AID": 2,
"Id": 5
}
```

Token (/token)

Methode	Pfad	Bedeutung	Auth
POST	/token	Erstellt ein Token und gibt ihn dem User zurück	user, admin

Beispiel:

POST /token/

Response

```
{  
  "access_token":  
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwiaXNjaXhwIjoxNzgyMDQxOTYxfQ.eyJy81Cm7EhFbYehk9jpnSfBqK-5NMtdTDmEUQhHsOUUs",  
  "token_type": "bearer"  
}
```

Rollenmodell

User

Kann alles außer Tracks löschen

Admin

Kann alles

4. Vergleich Planung ↔ Umsetzung

Geplant	Ergebnis	Bemerkung
Genre Endpunkt	ging nicht	Spotify API stellt dafür keinen Endpoint zur Verfügung

Geplant	Ergebnis	Bemerkung
User_settings	keine Verwendung	Der Spotify-Token konnte über das Nugget Packet geholt werden (Tabelle war deshalb unnötig)
Kein Stats Endpoint	Stats selber aggregiert	Wir haben die Statistiken für die jeweiligen Models in deren eigener Route abgefragt

5. KI-Unterstützung

Verwendete Tools

- ChatGPT
- Claude
- GitHub Copilot

Beispiele

Prompt

hallo claude ich will mit fastapi ein usermodel erstellen und dort ein password anlegen wie kann ich das hashen ?

Ergebnis

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

6. Projekttagbuch

Datum	Person	Tätigkeit
21.05.2026	Dominik	Repository, Grundstruktur, Projektbeschreibung

Datum	Person	Tätigkeit
21.05.2026	Jonas	Datenbankplanung, erste ERM/RM-Struktur
22.05.2026	Jonas	ERM/RM erweitert, Tabellenstruktur geplant
24.05.2026	Dominik	Backend-Grundstruktur, requirements, FastAPI-Setup
25.05.2026	Jonas	Datenbankmodelle und Tabellenentwurf
26.05.2026	Dominik	User-/Settings-Struktur, erste API-Grundlagen
27.05.2026	Jonas	Track-/Genre-/Stats-Konzept vorbereitet
28.05.2026	Dominik	Artist-/Playlist-/User-Router vorbereitet
30.05.2026	Jonas	Track Records, Datenbanklogik, SQL-Abfragen
31.05.2026	Dominik	Artists und Playlists erweitert
01.06.2026	Jonas	Stats-Endpunkte, Aggregationen, JOIN-Abfragen
02.06.2026	Dominik	User- und Playlist-Endpunkte ergänzt
03.06.2026	Jonas	CRUD-Endpunkte, Validierung, Fehlerbehandlung
08.06.2026	Dominik	Rollenmodell, Auth-Grundlagen, API-Anpassungen
10.06.2026	Jonas	Statistikfunktionen, Top-Tracks/Top-Genres
10.06.2026	Dominik	Artist-/Playlist-Details, JOIN-Responses
11.06.2026	Jonas	Refactoring, SQL-Optimierung, Helper-Funktionen
11.06.2026	Dominik	Logging, Fehlerbehandlung, Router-Anpassungen
13.06.2026	Jonas	API-Tests, Bugfixes, Response-Modelle
13.06.2026	Dominik	Dokumentation, Projektbeschreibung, Endpunktübersicht
14.06.2026	Domink	Token Route implementiert und in allen anderen eingebaut
14.06.2026	Jonas	Finale DBI-Fixes, Statistik- und Datenbanklogik
14.06.2026	Dominik	Finale API-Anpassungen, Dokumentation ergänzt
15.06.2026	Dominik	Letzte Änderungen, Merge, Abgabevorbereitung

7. BedienungsanleitungP

Installation

```
pip install -r requirements.txt
```

Starten

```
uvicorn src.main:app --reload
```

API-Dokumentation

```
http://localhost:8888/docs
```